Student's Companion Guide: Exploring Ramanujan's Mathematics with Python

Chaman Singh Verma

July 19, 2026

Welcome! This guide is a student-friendly companion to the formal paper on Srinivasa Ramanujan's mathematical discoveries.

While academic papers use advanced university-level mathematics, this guide is written to be accessible to high school students. It explains the core concepts using high-school algebra, visual examples, and simple Python code. By combining mathematics with programming, you can see these deep discoveries come to life on your own computer!

1 Topic Roadmap

The table below provides a quick reference to the topics covered in this guide, matching each mathematical concept with its Python implementation and real-world application.

Topic	Core Math Concept	Python Function	Real-World Application
1. Pi Series	Infinite series convergence	`ramanujan_pi`	High-precision constants
2. Partitions	Divisibility under mod arithmetic	`check_ramanujan_congruences`	Combinatorial counting
3. Identities	Euler partition bijections	`check_rogers_ramanujan_identity`	Algebra & representation theory
4. Fractions	Nested fraction stability	`demo_rr_continued_fraction`	Floating-point numerical tools
5. Divisors & HCN	Factors & primality exponents	`demo_divisors_and_hcn`	FFT data chunking in DSP
6. Mock Theta	Near-symmetric automorphic forms	`demo_mock_theta_like`	Black hole entropy in physics
7. Tau Function	Coefficients of modular forms	`demo_tau_function`	The Langlands Program

Table 1: Roadmap of Ramanujan's concepts explored in this guide.

—

2 Getting Started: How to Run the Code

All of the mathematical concepts discussed in this guide are implemented in the Python program `ramanujan_codes.py`.

To run the interactive demonstration on your computer:

1. Open your terminal (macOS/Linux) or Command Prompt (Windows).
2. Navigate to the folder containing the project files.
3. Execute the following command:

```
python3 ramanujan_codes.py
```

This will execute a suite of calculations and print out the numerical results explained below.

—

3 Ramanujan's Series for π

3.1 The Concept

You probably know that π (pi) is the ratio of a circle's circumference to its diameter, which is approximately 3.14159. Because π is an *irrational* number, its decimal expansion goes on forever without repeating.

Mathematicians use **infinite series**—formulas that add up an infinite list of fractions—to calculate π to millions of decimal places. Ramanujan discovered several extremely rapid formulas for calculating the reciprocal of π ($1/\pi$).

3.2 How it Works

Ramanujan's most famous series for $1/\pi$ is:

$$\frac{1}{\pi} = \frac{2\sqrt{2}}{9801} \sum_{n=0}^{\infty} \frac{(4n)!(1103 + 26390n)}{(n!)^4 396^{4n}}$$

- The symbol ! means factorial (e.g., $4! = 4 \times 3 \times 2 \times 1 = 24$).
- The symbol $\sum$ means "sum up" all terms as n goes from 0 to infinity.

Each term of this sum is a fraction. As n increases, the denominator grows incredibly fast because of the term 396^{4n}. This means each new term in the sum is tiny, adding exactly **8 correct decimal places** to the approximation of π!

3.3 Running in Python

In `ramanujan_codes.py`, the function `ramanujan_pi(terms)` computes this sum:

```
# Calculate pi using 3 terms of the series
pi_approx = ramanujan_pi(terms=3)
print(pi_approx)
# Output: 3.141592653589793238 4626... (highly accurate!)
```

—

4 Partition Numbers & Congruences

4.1 The Concept

A **partition** of a positive integer n is a way of writing n as a sum of positive integers, where the order of the terms does not matter. The **partition function** $p(n)$ counts the number of ways to do this.

For example, let's partition the number 4:

1. 4
2. $3 + 1$

3. $2 + 2$
4. $2 + 1 + 1$
5. $1 + 1 + 1 + 1$

There are exactly 5 ways, so $p(4) = 5$.

4.2 Ramanujan's Congruences (Divisibility Rules)

Ramanujan noticed that partition numbers grow extremely fast:

- $p(10) = 42$
- $p(50) = 204,226$
- $p(100) = 190,569,292$

Despite this rapid, irregular growth, Ramanujan discovered three beautiful divisibility rules (congruences) that hold true for *every* non-negative index k:

1. **Rule of 5:** $p(5k + 4)$ is always divisible by 5.
   - *Example ($k = 0$):* $p(4) = 5$ (divisible by 5)
   - *Example ($k = 1$):* $p(9) = 30$ (divisible by 5)
2. **Rule of 7:** $p(7k + 5)$ is always divisible by 7.
   - *Example ($k = 0$):* $p(5) = 7$ (divisible by 7)
3. **Rule of 11:** $p(11k + 6)$ is always divisible by 11.
   - *Example ($k = 0$):* $p(6) = 11$ (divisible by 11)

4.3 Running in Python

You can check these rules up to $n = 100$ by running the congruence check function in Python:

```
check_ramanujan_congruences(limit=100)
```

—

5 The Rogers–Ramanujan Identities

5.1 The Concept

Imagine you have a bag of numbers, and you want to partition a number n under two completely different rules:

- **Rule A:** You can partition n using any numbers, but the numbers you choose must differ by **at least 2** (e.g., if you use 3, you cannot use 4, but you can use 5).
- **Rule B:** You can partition n using any numbers, but you are **only allowed to use numbers ending in 1, 4, 6, or 9** (numbers congruent to $\pm 1 \pmod 5$).

Let's try this for $n = 9$:

- **Rule A Partitions:** (9), $(8+1)$, $(7+2)$, $(6+3)$, $(5+3+1)$ $\rightarrow$ **5 ways**
- **Rule B Partitions:** (9), $(6+1+1+1)$, $(4+4+1)$, $(4+1+1+1+1+1)$, $(1+1+1+1+1+1+1+1+1)$ $\rightarrow$ **5 ways**

Even though the rules are completely different, they produce the **exact same number of partitions**! The Rogers–Ramanujan identities state that this equivalence holds true for *every single integer* n forever.

5.2 Running in Python

In `ramanujan_codes.py`, we verify this by expanding both rules as polynomials and comparing the coefficients (representing the number of partitions) up to degree 40:

```
check_rogers_ramanujan_identity(max_deg=40)
# Outputs: Match = True for all terms!
```

—

6 Continued Fractions

6.1 The Concept

A **continued fraction** is a number written as a fraction nested inside the denominator of another fraction, going on forever:

$$R(q) = \cfrac{q^{1/5}}{1 + \cfrac{q}{1 + \cfrac{q^2}{1 + \cfrac{q^3}{1 + \ddots}}}}$$

Unlike normal infinite sums, continued fractions are incredibly stable. Computers use continued fractions to evaluate complex functions (like sines, cosines, or statistical curves) quickly and accurately.

6.2 Running in Python

You can watch the fraction converge as we calculate it deeper and deeper:

```
demo_rr_continued_fraction(q=0.2)
# Output:
# depth=  5 -> R(0.2) = 0.607833000...
# depth= 20 -> R(0.2) = 0.607849829...
# depth=100 -> R(0.2) = 0.607849829... (fully converged!)
```

—

7 Divisors & Highly Composite Numbers

7.1 The Concept

- **Divisors:** The divisors of a number are the integers that divide it evenly. For example, the divisors of 6 are $1, 2, 3$, and 6. The divisor count function $d(n)$ counts these divisors, so $d(6) = 4$.
- **Highly Composite Numbers (HCNs):** A highly composite number is a positive integer that has **more divisors than any number smaller than it**. You can think of them as the "most divisible" numbers.
  - 12 has 6 divisors $(1, 2, 3, 4, 6, 12)$. Since no number below 12 has 6 or more divisors, 12 is highly composite.

7.2 Real-World Application

Highly composite numbers are extremely useful in computer science, specifically for the **Fast Fourier Transform (FFT)** algorithm, which processes sound and image data. Dividing data into equal chunks is much faster if the total size of your dataset is a highly composite number (like 60 or 120) because it can be split in many different ways.

—

8 Mock Theta Functions

8.1 The Concept

A **mock theta function** is a complex mathematical function defined by an infinite series. They behave almost like modular forms—which have a high degree of symmetry—but they are missing a specific piece.

Ramanujan discovered them on his deathbed in 1920. He wrote about them in his final letter to G. H. Hardy, but could not formally define them. They remained one of the biggest mysteries in mathematics until 2002, when researchers finally understood their structure.

8.2 Black Hole Connections

In modern physics, mock theta functions are used in **string theory** to count the quantum states of energy (microstates) inside a **black hole**. This allows physicists to calculate black hole entropy, helping bridge the gap between Einstein's gravity and quantum mechanics.

—

9 The Ramanujan Tau Function

9.1 The Concept

The Ramanujan tau function $\tau(n)$ is a rule that assigns an integer to every number n. They appear as the coefficients of a special infinite product:

$$\Delta(q) = q(1-q)^{24}(1-q^2)^{24}(1-q^3)^{24}\cdots = \sum_{n=1}^{\infty} \tau(n)q^n$$

The first few values are $\tau(1) = 1$, $\tau(2) = -24$, $\tau(3) = 252$, and $\tau(4) = -1472$.

9.2 Multiplicativity

Ramanujan discovered that if you pick two coprime numbers m and n (meaning they share no common factors other than 1), then:

$$\tau(m \times n) = \tau(m) \times \tau(n)$$

Example: $\tau(2) \times \tau(3) = (-24) \times 252 = -6048$, which is exactly $\tau(6) = -6048$.

9.3 Running in Python

You can compute and verify these properties in Python:

```
demo_tau_function()
```

—

10 Lab Exercises, Experiments, and Explorations

This section contains a collection of computational experiments and mathematical problems designed to deepen your understanding. You can write small Python scripts or modify the functions inside `ramanujan_codes.py` to solve them.

10.1 Topic 1: Series for π

1. **Rate of Convergence Experiment:** Run the function `ramanujan_pi` with $1, 2, 3$, and 4 terms. Write a script to calculate the absolute error $|\text{ramanujan\_pi(t)} - \pi|$ at each step (using Python's `math.pi` as the reference). Verify that each term reduces the error by a factor of roughly 10^{-8}.

2. **The Chudnovsky Series Exploration:** Research the Chudnovsky algorithm, which was published in 1989 and is currently used to set world records for calculating π. How does it relate to Ramanujan's formula? Write a short paragraph explaining the structural similarities.

10.2 Topic 2: Partition Congruences

1. **Congruence Loop Experiment:** Write a Python loop that computes $p(n)$ up to $n = 200$, and prints out $p(5k+4) \pmod 5$, $p(7k+5) \pmod 7$, and $p(11k+6) \pmod{11}$ for all possible integers k. Verify that the remainder is always zero.

2. **Manual Partition Exploration:** List all the partitions of $n = 5$ by hand (there should be $p(5) = 7$ of them). Then, verify Ramanujan's modulo 7 congruence: since $5 = 7(0) + 5$, $p(5)$ must be divisible by 7. Confirm that the total number of partitions you wrote down is indeed divisible by 7.

10.3 Topic 3: The Rogers–Ramanujan Identities

1. **Partition Verification Experiment:** For $n = 5$, list all partitions that satisfy Rule A (parts differ by at least 2) and all partitions that satisfy Rule B (parts congruent to $\pm 1 \pmod 5$). Verify that both lists contain the exact same number of partitions.

2. **The Second Rogers–Ramanujan Identity Exploration:** The second Rogers–Ramanujan identity states that partitions into parts differing by at least 2 with no part equal to 1 are equinumerous to partitions into parts congruent to $\pm 2 \pmod 5$. Write a Python function to verify this identity up to $n = 20$.

10.4 Topic 4: Continued Fractions

1. **Speed of Convergence Experiment:** The convergence speed of the Rogers–Ramanujan continued fraction $R(q)$ depends heavily on q. Run `rogers_ramanujan_continued_fraction(q, depth)` for $q = 0.1, 0.5$, and 0.9 with depths from 1 to 20. Observe and record how the convergence rate slows down as q approaches 1.

2. **Golden Ratio Exploration:** A remarkable result discovered by Ramanujan is that when $q = 1$, the Rogers–Ramanujan continued fraction is directly related to the Golden Ratio $\phi = \frac{1+\sqrt{5}}{2}$ by the formula:
$$R(1) = \frac{\sqrt{5} - 1}{2} = \phi - 1$$
Evaluate $R(1)$ numerically using your continued fraction code for depth $d = 100$, and verify that it matches $\phi - 1 \approx 0.6180339887$.

10.5 Topic 5: Highly Composite Numbers

1. **Perfect Numbers Experiment:** A perfect number is a positive integer that is equal to the sum of its proper positive divisors (excluding the number itself). For example, $6 = 1 + 2 + 3$ is perfect. Write a Python function to find all perfect numbers under $10,000$. (Hint: Use the `sigma(n)` function!)

2. **Prime Exponent Exploration:** Verify Ramanujan's factorization theorem for Highly Composite Numbers: for every HCN, the prime exponents in its factorization must be non-increasing (e.g., in $n = 2^{a_2}3^{a_3}5^{a_5}\ldots$, we must have $a_2 \geq a_3 \geq a_5 \ldots$). Find the factorizations of $12, 24, 36, 48$, and 60, and verify this property.

10.6 Topic 6: Mock Theta Functions

1. **Maass Form Symmetries Exploration:** Research why mock theta functions are described as having "mock" modular symmetry. In a short paragraph, explain what a modular form is and why mock theta functions are considered to be modular forms that are "missing" a piece.

10.7 Topic 7: The Ramanujan Tau Function

1. **Non-Coprime Failure Experiment:** Ramanujan's multiplicativity states that $\tau(mn) = \tau(m)\tau(n)$ only when $\gcd(m, n) = 1$. Write a Python script to check what happens when $m = 2$ and $n = 2$ (which are not coprime). Calculate $\tau(4)$ and compare it to $\tau(2) \times \tau(2)$. Does the multiplicativity rule still hold?

2. **Tau Prime Power Recurrence Experiment:** For prime powers, the tau function satisfies the recurrence relation $\tau(p^r) = \tau(p)\tau(p^{r-1}) - p^{11}\tau(p^{r-2})$ for $r \geq 2$. Write a Python script to verify this relation for $p = 2, r = 2$ and $p = 2, r = 3$.

11 Hints and Selected Answers

11.1 Topic 1: Series for π

- **Hint (1):** In your script, import `math` and use `math.pi`. Loop over `terms = [1, 2, 3, 4]`, calculate the value using `ramanujan_pi(terms, precision=100)`, convert it to float via `float()`, and calculate the absolute difference.

- **Answer (2):** Both formulas represent $1/\pi$ as a sum of terms involving factorials, large powers in the denominator, and linear terms in n. The Chudnovsky series is based on similar modular transformations but uses 53360 and 640320^{3n} (based on the j-invariant of $\sqrt{-163}$), yielding 14 correct digits per term.

11.2 Topic 2: Partition Congruences

- **Hint (1):** Write a loop like:

```
p = partition_numbers(200)
for k in range(40):
    n5 = 5 * k + 4
    if n5 < len(p):
        print(f"p({n5}) mod 5 = {p[n5] % 5}")

```

  Repeat similarly for $7k + 5$ and $11k + 6$.

- **Answer (2):** The $p(5) = 7$ partitions are: (5), $(4 + 1)$, $(3 + 2)$, $(3 + 1 + 1)$, $(2 + 2 + 1)$, $(2+1+1+1)$, and $(1+1+1+1+1)$. The total count is 7, which is divisible by 7 (satisfying the congruence for $k = 0$).

11.3 Topic 3: The Rogers–Ramanujan Identities

- **Answer (1):** For $n = 5$, Rule A partitions are: (5), $(4+1) \to 2$ ways. Rule B partitions are: $(4 + 1)$, $(1 + 1 + 1 + 1 + 1) \to 2$ ways. Both lists have size 2.

- **Hint (2):** Use a logic similar to the sum-side builder but omit $n = 1$ factor in polynomial multiplication, or filter partitions in a custom loop.

11.4 Topic 4: Continued Fractions

- **Answer (1):** For $q = 0.1$, the value converges to 0.518625 within 5 iterations. For $q = 0.9$, the continued fraction converges much slower, requiring depth ≥ 50 to stabilize.

- **Answer (2):** Evaluating $R(1)$ at depth 100 yields approximately 0.618033988749, which matches $\phi - 1 = \frac{\sqrt{5}-1}{2}$.

11.5 Topic 5: Highly Composite Numbers

- **Hint (1):** Loop n from 1 to $10,000$. If `sigma(n) - n == n`, then n is perfect. (Selected answers: $6, 28, 496, 8128$).

- **Answer (2):** Factorizations: $12 = 2^2 \times 3^1$ (exponents: $2 \geq 1$), $24 = 2^3 \times 3^1$ (exponents: $3 \geq 1$), $36 = 2^2 \times 3^2$ (exponents: $2 \geq 2$), $48 = 2^4 \times 3^1$ (exponents: $4 \geq 1$), $60 = 2^2 \times 3^1 \times 5^1$ (exponents: $2 \geq 1 \geq 1$). In all cases, the prime exponents are non-increasing.

11.6 Topic 6: Mock Theta Functions

- **Answer (1):** A modular form is a function on the complex upper half-plane with a high degree of symmetry under modular transformations. Mock theta functions mimic these transformation properties, but they do not satisfy them exactly; they are missing a non-holomorphic part. Once completed by adding a specific integral term, they form a mock modular form.

11.7 Topic 7: The Ramanujan Tau Function

- **Answer (1):** For $m = 2, n = 2$, we have $\tau(4) = -1472$. However, $\tau(2) \times \tau(2) = (-24) \times (-24) = 576$. Since $-1472 \neq 576$, the multiplicativity rule fails when m and n are not coprime.

- **Answer (2):** For $p = 2, r = 2$: $\tau(4) = \tau(2)\tau(2) - 2^{11}\tau(1) \implies -1472 = 576 - 2048$, which is correct.